

Data Science Challenge

Truthfulness classification of public statements using large language models.

The Dataset

The dataset for this challenge consists of thousands of statements (quotes) made by politicians and other public figures. The data includes various information on each statement and on the person who made it. The truthfulness of each statement was also evaluated by expert human editors, who then assigned an appropriate truthfulness label.

All the data is included in the provided `data.csv` file. The file includes one comma-separated line for each statement, with the following attributes (columns):

- **label:** a truthfulness label assigned by human annotators. Possible values (from most to least truthful) are: `true`, `mostly-true`, `half-true`, `barely-true`, `false`, `extremely-false`.
- **statement:** the text of the actual statement that is being evaluated.
- **subjects:** a list of topics related to the statement, separated by a `$` sign — for example `state-financestaxes`.
- **speaker_name:** the name of the person who made the statement.
- **speaker_job:** the job title of the person who made the statement.
- **speaker_state:** the US state associated with the person who made the statement.
- **speaker_affiliation:** the political or professional affiliation of the person — for example `democrat`, `republican`, `journalist`.
- **statement_context:** the context in which the statement was made — for example `a radio interview`, `a press conference`.

The task is a **binary truthfulness classification** problem: each statement should ultimately be predicted as either **True** or **False**. It is up to you to decide how to map the six-way human label onto this binary target.

Challenge Deliverables

DELIVERABLE 1 — PYTHON PACKAGE

The first deliverable is a Python package that implements the components below, accompanied by brief instructions on how to install and use it. While it is up to you to decide how to structure your package, it should include at least the following.

Common requirements for all components. Each component must be able to **efficiently process a set of data points** (not just one) and return its outputs for all of them. In addition, for `predict()` and `explain()` endpoints, if the caller also supplies the **correct labels** for those points, the component must return **appropriate performance metrics** for its own predictions on that set.

1 Zero-shot LLM Predictor

Build a **pure-LLM predictor** that leverages the prior knowledge of an LLM to predict whether a statement is True or False. It simply prompts the LLM with the attributes of a given row and asks for an answer. **No fine-tuning and no training of any kind** at this stage.

ENDPOINT `predict(points, labels=None)`

- **Input:** `points` — a set of statements with their attributes (excluding the label); optional `labels` — the correct labels for those points.
- **Output:** a True/False prediction for each point. If `labels` are provided, it additionally returns performance metrics for its predictions on the set.

Prompt design, choice of model, and how much of the row's metadata you include are up to you.

2 Fine-tuned LLM Predictor

Fine-tune the LLM-based predictor to improve on the zero-shot baseline. This component has two endpoints — one to perform the fine-tuning and one for inference.

ENDPOINT 1 `fine_tune(training_dataset)`

- **Input:** `training_dataset` — a dataset of statements in the same format as `data.csv` (i.e. including labels).
- **Behavior:** performs the data preparation and train/validation split and runs the fine-tuning, producing a fine-tuned model (the heavy compute may run via a provider's fine-tuning API).
- **Output:** the resulting fine-tuned model, or a handle/identifier for it.

ENDPOINT 2 `predict(points, labels=None)`

- The **same signature and behavior as Component 1's `predict`**, but backed by the fine-tuned model, so the two predictors are interchangeable.

3 Explainer

Build an **explainer** that can work with **either** model (zero-shot or fine-tuned).

ENDPOINT `explain(model, points, labels=None)`

- **Input:** `model` — a predictor (Component 1 or Component 2); `points` — a set of statements with their attributes; optional `labels` — the correct labels for those points.
- **Output:** for each point, the **model's prediction** and an **explanation of the factors that drive that prediction**. If `labels` are provided, it additionally returns performance metrics for the underlying model's predictions on the set.

The nature of the explanations is up to you.

Second Deliverable — Presentation

DELIVERABLE 2 — SLIDES

A short presentation (no more than 10 slides) that describes your approach and main findings. This is where you explain and justify the decisions behind the code. It should cover:

- **Target framing:** how you mapped the six-way human label onto the binary True/False target, and why.
- **Evaluation methodology:** which metrics you chose and why (consider, for example, class balance, calibration, and the cost of different error types), and the held-out data and protocol you used to compare the two predictors fairly.
- **Zero-shot predictor:** your prompt and model design, and its baseline results.
- **Fine-tuned predictor:** your fine-tuning approach (data preparation, train/validation split, and key choices) and the improvements it produced over the zero-shot baseline.
- **Explainer:** how it works, why its explanations are meaningful and trustworthy for this kind of model, and what they reveal.

Evaluation Criteria

Your work will be evaluated based on:

- The quality of the results returned by your solution (predictive power of both predictors, the size and credibility of the fine-tuning improvement, and the usefulness of the explanations).
- The soundness of your evaluation methodology (appropriate metrics, fair baseline-vs-fine-tuned comparison).
- The quality and efficiency of your Python code.

How to Submit

You have **1 week** to submit your work. You can email your deliverables back to the same email address from which you received the challenge. Please do not share the challenge or your code in a public repository such as PyPI, GitHub, BitBucket, or any other similar website.

This challenge is meant to demonstrate the way you approach the problem and lay the foundations for a solid solution. We don't expect you to deliver a perfect solution in 1 week :-). Do your best in the time that you have, and feel free to contact us with questions at any point!

Private and Confidential · © Satalia